WSQ COURSE • TGS-2025052468

Version 16.0 · 9 Sep 2026

Agentic AI Applications with Claude Code

Tertiary Infotech Academy Pte Ltd
UEN: 201200696W

www.tertiarycourses.com.sg | enquiry@tertiaryinfotech.com | +65 6100 0613

Digital Attendance

Attendance is mandatory for SSG funding.

Take AM, PM and Assessment digital attendance for every session.

Use the SSG / TMS QR code provided by your trainer.

Minimum 75% attendance required

About the Trainer

[Trainer Name]

[Title / Role]

Qualifications: ______________________________

Expertise: ______________________________

Experience: ______________________________

Contact / Profile: ______________________________

INTRODUCTION

About the Trainer

Dr. Alfred Ang

AI Trainer & Engineer

PhD-qualified trainer specialising in AI, software engineering and developer tooling.

Hands-on experience building agentic AI applications with Claude Code.

Trains professionals across Singapore on practical, project-based AI skills.

Certifications: PMP, AWS, Microsoft, CompTIA, CKA, CKAD.

linkedin.com/in/angchewhoe

github.com/alfredang

INTRODUCTION

Let's Know Each Other

Take a minute to introduce yourself to the class:

- Your name & role
- What you do day to day
- Your coding experience
- What you want from this course

HOUSEKEEPING

Ground Rules

Phones on silent

Step out quietly for calls or breaks.

Participate actively

No question is stupid — ask freely.

Mutual respect

Agree to disagree; one conversation at a time.

Be punctual

Return from breaks on time.

75% attendance

Required for funding eligibility.

Stay hands-on

Follow along with every activity.

ROADMAP

Course Outline

1

Claude Code Fundamentals

- Setup & the Agentic Loop
- Context, Memory & Sessions
- Permissions & Scheduled Tasks
- **Activity: Build & deploy a website**

2

Tools and Commands

- Custom Commands
- MCP Tools
- Playwright MCP
- **Activities: /gitpush + MCP screenshot**

3

Skills, Agents & Hooks

- Agent Skills
- Sub Agents
- Hooks
- **Key Concepts Summary**

Lab Materials

Four labs, each in its own folder. They build on one another — Lab 1 creates the website that every later lab extends.

Lab	Topic	Description	Folder
Lab 1	Topic 1	The 7-step build workflow — goal, plan, execute, /init, GitHub, Pages, CI/CD.	`labs/lab1-seven-steps/`
Lab 1b	Topic 2	A /publish custom command + Playwright MCP screenshots into the README.	`labs/lab1b-commands-mcp/`
Lab 2	Topic 3	Install 3 skills — cybersecurity, kanban, UI/UX — and revamp the site.	`labs/lab2-skills/`
Lab 3	Topic 3	Hooks, sub agents and /loop — the event, delegation and schedule triggers.	`labs/lab3-hooks/`

TIP Every step gives you the exact prompt to paste, what you should see, and a checkpoint before moving on.

Lesson Plan (9:00 AM – 6:00 PM)

Time	Session
9:00 – 9:30	Welcome, admin, attendance (AM), introductions, ground rules
9:30 – 10:30	Topic 1.1: setup, agentic loop & tools, permission modes
10:30 – 10:40	*Morning break*
10:40 – 11:20	Topic 1.2: context, memory, sessions, scheduled tasks, /goal
11:20 – 12:30	Activity 1: Build & deploy a website
12:30 – 1:15	*Lunch break*
1:15 – 1:45	Topic 2: Custom commands · Activity 2: /gitpush
1:45 – 2:45	Topic 2: MCP tools · Activity 3: Playwright screenshot + form test
2:45 – 2:55	*Afternoon break*
2:55 – 3:40	Topic 3: Agent skills · Activity 4: frontend-design & SEO
3:40 – 4:25	Topic 3: Sub agents & hooks · Activity 5: security + UX agents
4:25 – 4:50	Activities 6–7 · hooks, testing, optimisation & deployment
4:50 – 5:00	Summary, Q&A, attendance (PM)
5:00 – 6:00	Final Assessment: 30 min WA + 30 min PP (on the LMS)

Briefing for Assessment

Clear your space

Place phones and other materials under the table or on the floor.

No recording

No photos or recording of assessment materials.

Open book

Only slides, learner guide and approved materials are allowed.

Work individually

Complete the assessment on your own; no discussion.

Manage your time

Written Assessment then Practical Performance — watch the clock.

Appeals

An appeal process is available if you disagree with the outcome.

Assessment & Funding

Final Assessment

- Written Assessment (WA) — 30 min, starts 5:00 PM
- Practical Performance (PP) — 30 min
- Format: Open book — class ends 6:00 PM
- Allowed: slides, learner guide & approved materials only
- Appeal process available

Criteria for Funding

Minimum 75% attendance

Based on SSG digital attendance records.

Assessed as 'Competent'

Complete and pass both assessment components.

LEARNING MANAGEMENT SYSTEM

Courseware & Assessment on the LMS

Download the courseware

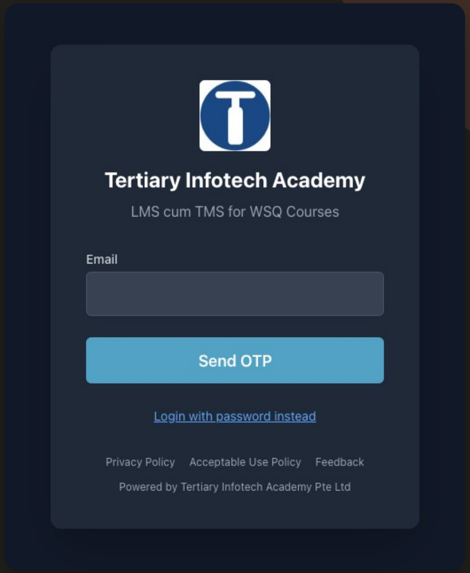
Get the slides and learner guide from the LMS for the open-book assessment.

Take the assessment

Complete the Written Assessment and Practical Performance on the LMS.

Log in

Use your enrolled account to access the course materials and assessment.



LMS: https://lms-tms.tertiaryinfotech.com/

TOPIC 1

Claude Code Fundamentals

1. Overview of Claude Code
2. The Agentic Loop
3. Context Engineering
4. Activity: Build a website

The Claude Product Family

Anthropic offers several Claude products — here's how they differ so you know which is which.

Claude.ai

The chat assistant on web, desktop & mobile for everyday tasks.

Claude Code

Agentic coding tool in your terminal, IDE & desktop.

Claude Cowork

Desktop agent for non-developers to automate files & tasks.

Claude Design

Generates polished UI & frontend designs.

Claude for Microsoft

Claude inside Microsoft 365 & Outlook.

Claude for Chrome

Browser agent that navigates and acts on the web for you.

Claude for Science

Speeds up research workflows for scientists and labs.

TIP *All are powered by the same Claude models — they differ in where they run and who they're for.*

TOPIC 1 · OVERVIEW

The Evolution of AI Engineering

Prompt Engineering

2023

Craft the wording of a single prompt to get good output.

Context Engineering

2024

Curate what the model sees — files, memory and tools.

Harness Engineering

2025

Build the system around the model: loop, tools, verification.

Harness Systems

2026

Agents run long tasks autonomously — Claude Code, Cowork, Codex.

More capable models shift the work from wording prompts → curating context → engineering the whole harness.

What is Harness Engineering?

A harness is the system built around the model — the agentic loop, tools, context management, verification and guardrails — that turns a capable model into an agent which can complete long-running tasks. Harness engineering is designing that scaffolding (and stripping it back as models improve).

A harness provides:

- The agentic loop
- Tool access (files, shell, MCP)
- Context & memory management
- Verification & guardrails

Examples of harness systems

Claude Code	Claude Cowork
OpenAI Codex	Gemini CLI
OpenClaw	Hermes Agent

TIP *Build the simplest harness that works — and re-examine it with every new, more capable model.*

What is Claude Code?

Claude Code is an agentic coding tool that lives in your terminal. You describe a goal in plain English, and Claude plans, writes, runs and verifies the work — editing files, running commands and using tools on your behalf.

Terminal-native

Runs where you already work — no heavy IDE required.

Agentic

Plans multi-step tasks and acts, not just autocompletes.

Tool-using

Reads/writes files, runs bash, calls MCP servers.

Extensible

Custom commands, skills, sub-agents and hooks.

What Claude Code Can Do

Claude Code handles real engineering tasks end to end — describe the outcome and it does the work.

Build

Create features, apps and whole projects from a prompt.

Debug & fix

Find and fix errors; read logs and stack traces.

Refactor

Improve, restructure and document existing code.

Automate

Run git, tests, deployment and CI/CD workflows.

Common use cases: marketing sites & landing pages · internal web tools · full-stack apps · mobile apps · scripts & automations · data tasks.

Setup Claude Code

The same agentic loop runs everywhere — choose the interface that fits how you work.

Terminal

Install the CLI, then run `claude` in any project.
Mac/Linux: curl -fsSL claude.ai/install.sh | bash

IDE (VS Code)

Install the Claude Code extension (also Cursor & JetBrains): inline diffs, @-mentions, plan review.

Desktop App

A full GUI for parallel sessions, scheduled tasks and connectors — runs on your machine.

TIP No CLI yet? In VS Code just ask Claude Code: "install claude cli in the terminal for me."

Setup Claude Code in VS Code

Add Claude Code to VS Code in five steps — from a clean install to signed in and ready to build.

1. **Install VS Code**
   Download from code.visualstudio.com/download and install it for your operating system.
2. **Open a project folder**
   Launch VS Code and open an empty folder to start a new project.
3. **Install the Claude Code extension**
   In the Extensions panel, search for "Claude Code" and click Install.
4. **Move it to the secondary side bar**
   Drag the Claude Code icon into the secondary side bar so it stays open beside your editor.
5. **Sign in with your subscription**
   Click Sign in and log in with your Claude Pro or Max subscription — you're ready to build.

TIP No CLI needed to start — the VS Code extension runs the same agentic loop. Ask it to "install the Claude CLI" anytime.

Claude Code Pricing

Claude Code is included in the subscription plans (from Pro up) and is also available pay-as-you-go via the API.

Pro

$17/mo (annual) · $20 monthly. Claude Code included.

Max

From $100/mo — 5× or 20× more usage than Pro.

Team

$20/seat standard · $100/seat premium (5× usage).

Enterprise

$20/seat + usage at API rates. Contact sales.

API pay-as-you-go (per million tokens, input / output): Opus 4.8 $5 / $25 · Sonnet 4.6 $3 / $15 · Haiku 4.5 $1 / $5

TIP A free plan is also available. Prices as of June 2026 — verify at claude.com/pricing.

The Agentic Loop

Every Claude Code task runs the same repeating cycle. Understanding this loop is the key to working effectively with an agent.

1. Gather Context

Read files, run searches, use tools to understand the task.

2. Take Action

Edit code, run commands, call MCP tools to make progress.

3. Verify Work

Run tests, lint, view output — check the result is correct.

4. Repeat

Feed results back as new context and loop until the goal is met.

Goal in → context → action → verification → repeat → goal met

Why the loop matters

Give direction, not micro-steps

State the goal and constraints clearly; let the loop figure out the steps.

Verification is everything

An agent is only as good as its ability to check its own work — provide tests, linters, screenshots.

Steer mid-loop

Review the plan, interrupt, and course-correct rather than waiting for a wrong final answer.

Context is the fuel

What the agent can see determines what it can do — leading us to context engineering.

Built-in Tools

Tools are what make Claude Code agentic. Each tool use returns information that feeds the next decision in the loop.

File ops

Read, edit, create, rename and reorganise files.

Search

Find files by pattern, grep content, explore code.

Execution

Run shell commands, tests, builds and git.

Web

Search the web and fetch docs or error messages.

TIP *Plus subagents, code intelligence and your configured extensions (MCP, skills, hooks).*

The Agentic Loop in Action

You: *"Build a responsive software-dev website with an enquiry form, then test it."*

1. Gather context

Reads the folder & CLAUDE.md to understand the existing site and conventions.

2. Plan

Drafts the approach — hero, testimonials and enquiry form sections — for you to approve.

3. Take action

Writes index.html, styles.css and script.js with form validation.

4. Verify work

Opens the page via Playwright MCP and submits the form to test it.

5. Repeat

Spots a validation bug, fixes script.js, re-tests — then reports done.

7 Steps to Build an App with Claude Code

1

Goal & Features

Define what to build (draft the prompt with ChatGPT).

2

Plan Mode

Let Claude plan before coding (Shift+Tab).

3

Execute

Run the plan in Bypass Permissions mode.

4

Context (/init)

Generate CLAUDE.md to capture project context.

5

Commit to GitHub

Version-control and push your code.

6

Deployment

Publish the app (e.g. GitHub Pages).

7

CI/CD

Automate build & deploy with GitHub Actions.

TIP *We'll do steps 1–3 now (Activity 1a), then context engineering, then steps 4–7 (Activity 1b).*

Start With a Clear Prompt

- **Describe the goal & features**

The clearer the brief, the better the build.

- **Use ChatGPT to draft it**

Ask ChatGPT to expand your idea into a detailed prompt, then paste it into Claude Code.

- **Cover the essentials**

Goal · sections · behaviour · styling · deliverable.

```
Example prompt (software-dev website)

Build a single-page, responsive website
for a software development company
(HTML, CSS, vanilla JS — no frameworks).

Goal: market services + capture leads.

Sections: 1) Hero + CTA  2) 3 testimonial
cards  3) Enquiry form (JS validation,
console.log, commented fetch()).

Styling: mobile-first, CSS variables,
Google Font, smooth scroll, accessible.

Deliverable: runs by opening the file.
```

TIP The full prompt is in your Learner Guide (Activity 1).

Permission Modes

Permission modes control how often Claude pauses to ask before editing files or running commands.

`default`

Reads freely; asks before edits and shell commands.

`acceptEdits`

Auto-approves file edits and common filesystem commands.

`plan`

Explores and proposes a plan without editing source.

`auto`

Runs with background safety checks (research preview).

TIP *Press Shift+Tab to cycle modes; the current mode shows in the status bar.*

Bypass Permissions — Handle with Care

`bypassPermissions` / `--dangerously-skip-permissions`

- Skips all permission prompts and safety checks — tool calls run immediately.
- Offers NO protection against prompt injection or unintended actions.
- Use only in isolated containers / VMs without internet access.
- Refuses to start as root/sudo; cannot be entered mid-session.
- **Prefer auto mode for fewer prompts WITH background safety checks.**

Bypass Permissions in VS Code

To let Claude build hands-free, enable bypass-permissions mode — use it only in a trusted, sandboxed project.

1. Open VS Code Settings → Extensions → Claude Code.
2. Enable “Allow dangerously skip permissions”.
3. In the prompt box, open the mode selector.
4. Choose “Bypass permissions”.
5. Give your build prompt — Claude runs without per-action prompts.

```
CLI equivalent

$ claude \
  --dangerously-skip-permissions
```

⚠ *No safety checks or prompt-injection protection. Sandboxed / trusted projects only — prefer auto mode otherwise.*

ACTIVITY 1A

Goal, Plan & Build (Steps 1–3)

1. Step 1 — Goal & Features: draft a detailed build prompt (use ChatGPT) for the software-dev website.
2. Step 2 — Plan Mode: have Claude plan the build (Shift+Tab) and review the plan.
3. Step 3 — Execute: approve the plan and build the site in Bypass Permissions mode.
4. Open index.html and check the hero, testimonials and enquiry form work.

TIP *Full website prompt is in your Learner Guide (Activity 1).*

Context Engineering

Context engineering is the practice of curating exactly what Claude sees — instructions, files and tools — so it has the right information and nothing that wastes its limited context window.

`CLAUDE.md`

Project memory loaded every session. Run /init to generate it.

`@ mentions`

Pull specific files or folders into context on demand.

`/compact`

Summarise the conversation to reclaim context space.

`/clear`

Wipe the slate for a fresh, unrelated task.

Managing Your Context

- **Keep CLAUDE.md tight**

Project overview, conventions, commands to run. Short and high-signal.

- **Pull only what's needed**

Use @file mentions instead of pasting whole folders.

- **Compact before it overflows**

Watch the context meter; /compact at natural breakpoints.

- **Clear between tasks**

Start fresh so old context doesn't bleed into new work.

```
CLAUDE.md

# My Web App

## Stack
Vanilla HTML, CSS, JS. No frameworks.

## Conventions
- Single index.html, mobile-first
- Use CSS variables for colors

## Commands
- Deploy: push to main (GH Pages)
```

Example CLAUDE.md (/init)

Run /init to generate a CLAUDE.md so Claude remembers your project's structure and conventions every session.

```
CLAUDE.md

# CLAUDE.md

## What this is
Single-page marketing site. HTML + CSS + vanilla JS — no framework, no build step.

## Running
Open index.html directly in a browser. No dev server.

## Architecture
- index.html  — sections: #home, #services, #testimonials, #contact
- styles.css  — design tokens in :root variables; mobile-first
- script.js   — enquiry-form validation (one DOMContentLoaded handler)
```

TIP *Keep CLAUDE.md short and high-signal — see github.com/alfredang/softwaredevleopment.*

The Context Window

Claude's context window holds everything it can see at once. As you work it fills up, so manage it deliberately.

What fills it

System prompt, CLAUDE.md, memory, conversation, file reads, tool output, skills.

/context

See what's using space; run /mcp for per-server token cost.

Auto-compaction

Near the limit, old tool outputs clear first, then history is summarised.

Stay lean

MCP schemas load on demand; subagents keep their own context.

TIP Put persistent rules in CLAUDE.md — conversation detail can be lost during compaction.

Memory Files

Each session starts fresh. Two mechanisms carry knowledge across sessions.

CLAUDE.md

- You write it — instructions & rules.
- Generate with /init; keep under ~200 lines.
- Scopes: project, user, or organisation.
- Loaded in full every session.

Auto Memory

- Claude writes it — learnings & patterns.
- Stored in MEMORY.md per repository.
- First 200 lines / 25KB load each session.
- Browse & edit with /memory.

Manage Sessions

A session is a saved conversation tied to a project directory. Resume, branch, name and switch between them.

`--continue`

Resume the most recent session in this directory.

`--resume`

Open the session picker to browse and search.

`/rename`

Name a session so it's findable later.

`/branch`

Fork the conversation to try another approach.

TIP Sessions are saved locally; use /clear and /compact to manage context within one.

Scheduled Tasks

Scheduled tasks (Desktop Routines) start a new session automatically — for daily reviews, audits or briefings.

Create

Routines → New routine → Local; set name & instructions.

Schedule

Hourly, daily, weekdays or weekly — or describe it.

Runs locally

Fires while the app is open and your machine is awake.

Manage

Run now, pause, edit, and review run history.

TIP *Just ask: "set up a daily code review that runs every morning at 9am."*

Keep Working Toward a Goal

/goal sets a completion condition. After each turn a fast model checks it, and Claude keeps working until it's met.

`/goal <cond>`

Set a verifiable end state, e.g. “all tests pass”.

`/goal`

Check status: turns, tokens and the latest reason.

`/goal clear`

Remove an active goal before it's met.

How

A Haiku evaluator judges the condition each turn.

TIP *Write conditions Claude can prove in the transcript — e.g. “npm test exits 0”.*

ACTIVITY 1B

Context, Deploy & CI/CD (Steps 4–7)

1. Step 4 — Context: run /init to generate CLAUDE.md for your project.
2. Step 5 — Commit to GitHub: initialise git and push your code.
3. Step 6 — Deployment: deploy to GitHub Pages via GitHub Actions; add a README.
4. Step 7 — CI/CD: confirm each push automatically rebuilds and redeploys the site.
5. Present your live GitHub Pages link: https://<username>.github.io/software-dev-site/

TIP Share your deployed GitHub Pages URL with the class.

TOPIC 2

Tools and Commands

1. The .claude directory
2. Custom Commands
3. MCP Tools
4. Playwright MCP

The .claude Directory

Claude Code keeps its configuration in two places: your user folder (applies everywhere) and the project folder (shared with your team via version control).

```
~/.claude/   (user — all projects)

settings.json   # settings, hooks
CLAUDE.md       # personal memory
agents/  skills/  commands/
projects/<p>/ # sessions + memory/
plugins/
~/.claude.json # MCP servers, config
```

```
<project>/.claude/   (team — in git)

settings.json        # shared settings
settings.local.json  # personal
CLAUDE.md  rules/ # memory & rules
commands/  # /slash commands
skills/<name>/SKILL.md
agents/<name>.md # sub agents
.mcp.json  # project MCP servers
```

TIP *User scope applies to every project; project scope is committed so your whole team shares it.*

User Level vs Project Level

Every customisation — commands, MCP tools, skills, CLAUDE.md and hooks — can be stored at two levels. User level applies to ALL your projects; project level applies only to that ONE project.

User Level · ~/.claude/

APPLIES TO ALL PROJECTS

- **Commands** ~/.claude/commands/<name>.md
- **MCP Tools** ~/.claude.json · claude mcp add -s user ...
- **Skills** ~/.claude/skills/<name>/SKILL.md
- **CLAUDE.md** ~/.claude/CLAUDE.md (your personal memory)
- **Hooks** ~/.claude/settings.json → "hooks"

Project Level · <project>/

THIS PROJECT ONLY

- **Commands** <project>/.claude/commands/<name>.md
- **MCP Tools** <project>/.mcp.json · claude mcp add -s project ...
- **Skills** <project>/.claude/skills/<name>/SKILL.md
- **CLAUDE.md** <project>/CLAUDE.md (shared with the team)
- **Hooks** <project>/.claude/settings.json → "hooks"

TIP *Keep personal preferences at user level; commit team-shared rules, commands, skills and hooks at project level. Both levels load together — where they conflict, project settings take precedence.*

Custom Slash Commands

Custom commands are reusable prompts saved as Markdown files. Type a slash command to run a multi-step workflow the same way every time.

- **Where they live**

.claude/commands/ in your project (or ~/.claude for personal).

- **How to make one**

Create name.md — the filename becomes /name.

- **Reload**

Restart Claude Code to pick up new commands.

- **Arguments**

Use $ARGUMENTS to pass input into the command.

```
.claude/commands/gitpush.md

Push all code to GitHub, then:
1. Create a README
2. Enable GitHub Pages
3. Update repo About + Pages link
4. Scan for secrets before publishing

Target: $ARGUMENTS
```

ACTIVITY 2

Create a /gitpush Command

1. Create .claude/commands/gitpush.md in your project.
2. Step: push/update all code to GitHub.
3. Step: generate a README for the repository.
4. Step: create a GitHub Page via GitHub Actions.
5. Step: update the repo About section and add the GitHub Pages link.
6. Step: security scan so no sensitive data is published.

TIP Restart Claude Code, then run /gitpush to execute the whole workflow.

What is MCP?

The Model Context Protocol (MCP) is an open standard that lets Claude Code connect to external tools and data sources through a common interface — like a USB-C port for AI.

MCP Server

Exposes tools/data (e.g. Playwright, GitHub, databases).

MCP Client

Claude Code connects to servers and calls their tools.

Transport

stdio or HTTP/SSE — transport-agnostic communication.

Tools

Each server publishes callable actions Claude can use.

Connect an MCP Server

- **Add a server**

claude mcp add — register an HTTP or local stdio server.

- **Pick a scope**

local (this project), project (.mcp.json, shared), or user (all projects).

- **Verify & use**

claude mcp list to check status; /mcp to manage in-session.

```
terminal

# hosted server
$ claude mcp add --transport \
   http docs <url>

# local stdio server (project)
$ claude mcp add playwright \
   -s project -- npx -y \
   @playwright/mcp@latest

$ claude mcp list
```

TIP *Project-scoped servers live in .mcp.json — commit it so teammates get the same tools.*

Playwright MCP

The Playwright MCP server lets Claude Code drive a real browser — open pages, click, fill forms and capture screenshots — perfect for testing the website you just built.

- **Install at project level**

Add the server to your project's MCP config.

- **Approve permissions**

Allow Claude to launch and control the browser.

- **Use the tools**

Ask Claude to open your site and take a screenshot.

terminal

```
$ claude mcp add playwright \
    npx @playwright/mcp@latest

> open my site and take a
  screenshot, then add it
  to the README
```

ACTIVITY 3

Install & Use Playwright MCP

1. Install the Playwright MCP tool at the project level.
2. Approve the browser permissions for Claude Code.
3. Ask Claude to open your deployed website in the browser.
4. Capture a screenshot of the site using the Playwright MCP tool.
5. Add the screenshot to your README and push to GitHub.

TIP *Verify the screenshot renders correctly in your GitHub README.*

ACTIVITY 3B

Test the Enquiry Form (Playwright)

1. Wire the enquiry form to send submissions to angch@tertiaryinfotech.com (e.g. Formspree or a fetch to your endpoint).
2. Ask Claude to use the Playwright MCP tool to open your deployed site.
3. Fill the form with test data (name, email, message) and submit it.
4. Confirm the success message appears and the data is captured in the console.
5. Check that the enquiry email actually arrives in the inbox.

TIP *Re-run after each change so every enquiry reliably reaches your email.*

TOPIC 3

Skills, Agents & Hooks

1. Agent Skills
2. Sub Agents
3. Hooks
4. Key Concepts Summary

Agent Skills

Skills are reusable folders of instructions (SKILL.md) plus optional scripts and resources that teach Claude how to do a specialised task. Claude loads a skill only when it's relevant.

SKILL.md

Name + description + instructions Claude follows on demand.

Project skills

.claude/skills/ — shared with your repo & team.

Install

npx skills add <repo> --skill <name>.

Customise

Edit the skill to fit your project's needs.

SKILL.md vs CLAUDE.md

Both are Markdown instruction files — the difference is WHEN Claude loads them.

SKILL.md

Loaded ON DEMAND

- **Trigger** — Claude reads the name + description, and loads it only when the task matches.
- **Scope** — One specialised capability — a workflow, a format, a house standard.
- **Cost** — Costs no context until it fires; you can keep dozens installed.
- **Lives in** — .claude/skills/<name>/SKILL.md

CLAUDE.md

Loaded EVERY SESSION

- **Trigger** — Read automatically at the start of every session — always in context.
- **Scope** — Project-wide facts: stack, conventions, commands, do-nots.
- **Cost** — Spends context on every turn, so keep it tight and high-value.
- **Lives in** — CLAUDE.md at the project root (or ~/.claude/CLAUDE.md)

TIP *Put stable project facts in CLAUDE.md; put specialised, occasional procedures in a skill.*

ACTIVITY 4

Install & Run Skills

1. Install frontend-design: npx skills add https://github.com/anthropics/skills --skill frontend-design
2. Customise the frontend design to appeal to software-dev clients.
3. Install seo-audit: npx skills add https://github.com/coreyhaines31/marketingskills --skill seo-audit
4. Run the seo-audit skill to optimise the website's SEO.
5. Commit the improvements and push to GitHub.

TIP Both skills install into .claude/skills/ at the project level.

Sub Agents

Sub agents are specialised assistants with their own prompt, tools and context window. Delegate focused jobs — like security review or UX critique — to keep the main thread clean.

- **Create with /agents**

Choose Project to save under .claude/agents/.

- **Give it a clear role**

Describe purpose, tools and when to invoke it.

- **Isolated context**

Each sub agent runs in its own context window.

```
.claude/agents/security-scan.md

---
name: security-scan
description: Scan for vulns
  & hardening
---
You are a security reviewer...
```

Why Use Sub Agents?

Reach for a sub agent when a side task would flood your main conversation with output you won't reference again.

Preserve context

Exploration & logs stay in the agent's own window — only the summary returns.

Enforce constraints

Limit exactly which tools the sub agent is allowed to use.

Reuse & specialise

Reuse configs across projects; focused prompts for each domain.

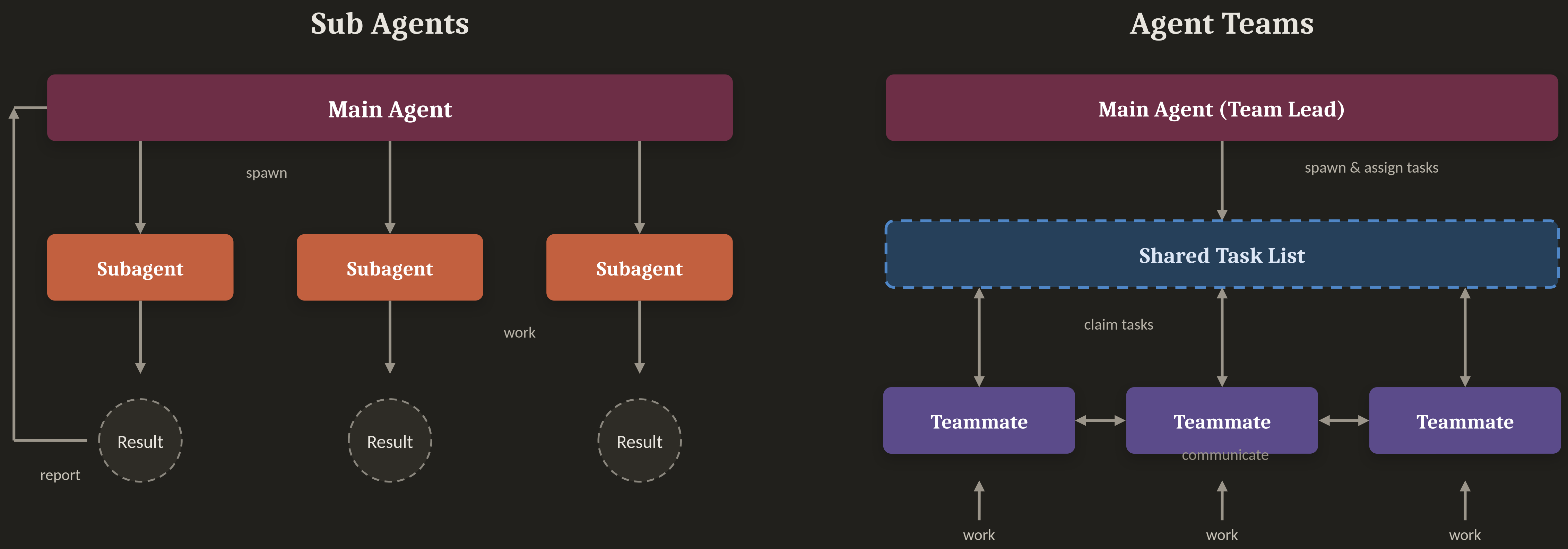
Control costs

Route narrow tasks to a faster, cheaper model.

TIP Define a custom sub agent when you keep spawning the same kind of worker with the same instructions.

TOPIC 3 · SUB AGENTS

Sub Agents vs Agent Teams



Sub agents report back to one main agent · agent teams coordinate via a shared task list.

ACTIVITY 5

Create Custom Sub Agents

1. Create a security-scan sub agent in .claude/agents/ to find vulnerabilities and harden the site.
2. Run it and apply the recommended fixes.
3. Install lead-magnets: npx skills add https://github.com/coreyhaines31/marketingskills --skill lead-magnets
4. Create a UI/UX review sub agent that uses lead-magnets to optimise for lead capture.
5. Apply improvements and push to GitHub.

TIP Both agents live in .claude/agents/ at the project level.

Hooks

Hooks are shell commands that run automatically at defined points in Claude Code's lifecycle — letting you enforce rules, format code, or trigger actions without asking.

`PreToolUse`

Runs before a tool — validate or block an action.

`PostToolUse`

Runs after a tool — auto-format or test.

`Notification`

React when Claude needs input or finishes.

`Stop`

Run cleanup or checks when a task ends.

Configured in .claude/settings.json under "hooks".

Automating with Hooks

- **Deterministic, not optional**

Hooks always run at their event — they don't rely on the model deciding to.

- **Common uses**

Auto-format after edits, run tests, block risky commands, send notifications.

- **Where**

Add a "hooks" block to .claude/settings.json (matchers + commands).

```
.claude/settings.json

{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command":
          "npx prettier --write ."
      }]
    }]
  }
}
```

TIP *Hooks receive event context on stdin as JSON — perfect for lint, format and guardrail scripts.*

ACTIVITY 6

Hooks — Floating WhatsApp Widget

1. Add a floating WhatsApp widget to the bottom-right of the website from Activity 1.
2. Style it as a round WhatsApp button that expands into a chat panel on click.
3. Add a Claude Code hook that fires 10 seconds after page load to auto-open the widget.
4. On auto-open, prompt the user “Hi! How can I help you?” with suggested queries as quick-reply chips.
5. Add voice activation — let the visitor speak their question using the Web Speech API.
6. Use the Playwright MCP to test the 10-second auto-open, the suggested queries and the voice prompt.

TIP Hooks live in .claude/hooks + .claude/settings.json; the widget behaviour lives in script.js.

ACTIVITY 7

Test, Optimise & Deploy

1. Use Playwright MCP to test the enquiry form end-to-end.
2. Ensure all enquiries are sent to angch@tertiaryinfotech.com.
3. Add a WhatsApp floating widget (bottom-right) with suggestive queries, linked to +65 9698 3731.
4. Run a final security scan and lead-magnet UX review.
5. Deploy the final version and verify the live site end-to-end.

TIP Record the live URL and the tests completed for your final verification.

What Triggers What

Six ways to extend Claude Code — each one is loaded by a different trigger. Knowing which fires when is how you choose the right mechanism.

CLAUDE.md	EVERY SESSION	Project memory — read automatically at the start of every session, so it is always in context.
Skills	ON DEMAND	SKILL.md folders — Claude reads the name + description, and loads one when the task matches.
Tools / MCP	ON DEMAND	Built-in and MCP tools — invoked by Claude during the agentic loop whenever the job needs them.
Slash Commands	MANUALLY	Saved prompts in .claude/commands/ — they run only when you type /<name> yourself.
Hooks	BY EVENT	Shell commands in settings.json — fired deterministically by a lifecycle event, not by the model.
Sub Agents	BY DELEGATION	Specialised assistants with their own context — spawned when the main agent delegates a job.

TIP Always-on context is expensive — keep CLAUDE.md tight and push specialised, occasional procedures into skills, commands and sub agents.

WRAP UP

Summary & Q&A

Topic 1	The agentic loop and context engineering power everything Claude Code does.
Topic 2	Custom commands and MCP tools automate and extend your workflow.
Topic 3	Skills, sub agents and hooks make Claude a specialised teammate.
Integrated Build	You built, deployed, tested, secured and optimised a real website.

ASSESSMENT

Assessment Flow

At the assessment stage, follow these five steps in order.

STEP 1

TRAQOM

Scan the TRAQOM QR code on the LMS and complete the survey.

STEP 2

Attendance

Take the assessment digital attendance.

STEP 3

Assessment

Complete the WA & PP.

STEP 4

Submit Answers

Submit your assessment answers on the LMS.

STEP 5

Sign the Record

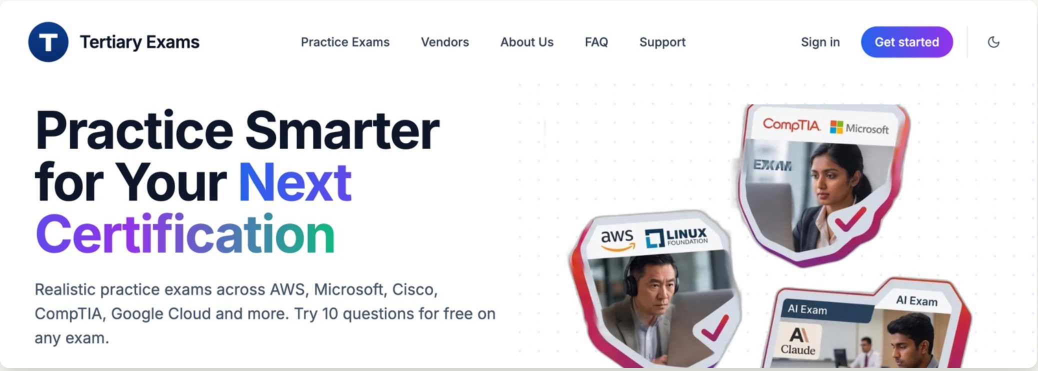
Sign your Assessment Summary Record.

TIP Courseware and the assessment are on the LMS: https://lms-tms.tertiaryinfotech.com/

Practice Exam Portal

Keep practising after class — Tertiary Exams offers realistic practice exams across AWS, Microsoft, Cisco, CompTIA, Google Cloud and more. Try 10 questions free on any exam.

1. Go to exams.tertiaryinfotech.com and search for your certification.
2. Try 10 free questions on any exam, in Practice Mode or Exam Mode.
3. Review your answers and explanations, then repeat until you are ready.



Practice Exams: https://exams.tertiaryinfotech.com/

TIP *Practice exams are the fastest way to find the gaps before the real certification.*

BEFORE YOU GO

Certification & TRAQOM Survey

TRAQOM Survey (Mandatory)

Complete the certification & TRAQOM survey to receive your certificate.

ai-lms-tms.tertiaryinfo.tech

Digital Attendance

Remember to take AM, PM and Assessment digital attendance — required for your funding.

Thank You!

Questions? We're here to help — during and after class.

enquiry@tertiaryinfotech.com | +65 6100 0613